SMARTCITY AI - PLATFORM TECHNICAL ARCHITECTURE

Gorakhpur Smart City Operations & Intelligent Municipal Operating System

Version: 2.5.0 Enterprise | Node.js Express 5 + MySQL 8.0 + Python FastAPI ML + Leaflet GIS + Socket.IO

Target Deployment: Municipal Corporation Gorakhpur (Nagar Nigam) & ICCC Command Center

1. SYSTEM ARCHITECTURE & TOPOLOGY

SMARTCITY AI is designed around a high-concurrency, Single Source of Truth municipal architecture. It integrates citizen portals, staff workstations, IoT sensor telemetry, and administrative control under a unified event-driven backend.

* Express 5 Gateway (Port 5000): High-throughput Node.js API server handling 29 REST route modules, Gzip/Deflate zlib payload compression (70-85% bandwidth reduction), JWT token lifecycle, and role-based access control.

* Socket.IO Real-Time Engine (Port 5000): WebSocket server managing active rooms (ambulance-tracking, parking-updates, emergency-alerts, water-updates, waste-updates, city-updates) with real-time push daemons.

* FastAPI ML Intelligence Service (Port 8000): Asynchronous Python 3.13 microservice hosting Scikit-Learn Random Forest Regressors, Gradient Boosting Classifiers, and Tfidf/Logistic Regression clinical NLP models.

* MySQL 8.0 Enterprise Relational Store: 117 active tables with connection pooling (max 20 concurrent connections), storing users, sensors, hospital beds, parking bays, traffic junctions, and immutable audit logs.

* Dual GIS Spatial Engine: Leaflet 1.9.4 and OpenLayers 9.2.4 running in client browsers, supporting custom GeoJSON layers, vector styling, and dynamic GPS vehicle radar tracking.

* IoT Hardware & Firmware Tier: ESP32 microcontrollers deployed with ultrasonic depth sensors (HC-SR04), PIR footfall detection (HC-SR501), and WiFi REST telemetry ingestion.

2. MASTER ALGORITHM CATALOG (FORMULAS, MATH & CODE LOCATION)

The platform incorporates 12 production-grade deterministic and machine-learning algorithms across municipal domains:

* 1. Webster Signal Optimization (IRC:93): Calculates optimum cycle length C0 and green phase splits for 42 junctions.

```
Formula: C0 = (1.5L + 5) / (1 - Y)
L = Total lost time = (lost_per_phase * num_phases) + all_red = 12s
Y = Sum of critical flow ratios (q_i / s_i), s_i = 1800 vph saturation flow
Effective Green Split: g_i = (y_i / Y) * (C0 - L)
Safety Constraints: 45s <= C0 <= 150s, Min Green = 15s, Max Green = 120s, Yellow = 4s
Location: backend/services/traffic_engine.js | backend/services/traffic_ai_service.js
```

* 2. Topological Graph Dijkstra Pathfinding: Calculates 3 real route alternatives across Gorakhpur road network (17 nodes, 18 arterial links).

```
Graph G = (V, E), Adjacency list with live junction congestion weights
Cost Function 1 (Fastest Arterial): Time = Distance / (SpeedLimit * (1 - Congestion/100 * 0.7))
Cost Function 2 (Low-Traffic Bypass): Cost = BaseTime * (1 + ((max(0, Congestion - 30)/15)^2.2))
Cost Function 3 (Min-Junction Route): Cost = BaseTime + (isJunction ? 4.5 mins : 0.5 mins)
Location: backend/services/gorakhpur_road_graph.js
```

* 3. Geodesic Haversine Spherical Distance: Measures true earth-curvature distance between vehicles and intersections for emergency preemption.

```
a = sin^2(dLat/2) + cos(lat1)*cos(lat2)*sin^2(dLon/2)
c = 2 * atan2(sqrt(a), sqrt(1-a))
Distance d = R * c  (Earth Radius R = 6,371,000 meters)
Green Wave Trigger: When Ambulance distance d <= 600m to approaching junction
Action: Interlocking lock opposing cross streets to Red; lock transit axis Green for 180s
Location: backend/services/traffic_ai_service.js | backend/services/ambulance_corridor_service.js
```

* 4. Waste Collection TSP Nearest-Neighbor: Solves Traveling Salesperson Problem for municipal compactor trucks collecting overflowing dustbins.

```
Heuristic: Greedy Nearest Neighbor across bins with fill_level >= 70%
Estimated Route Time = Sum(Collection_Time_per_bin [8 min] + Transit_Time [3.5 min/km])
Location: backend/services/waste_ai_service.js | backend/controllers/waste.controller.js
```

* 5. Hydraulic Leak & Pipe Burst Anomaly Detection: SCADA water loss and pressure gradient evaluation.

```
Loss Rate Pct = ((Inflow_Rate_LPS - Outflow_Rate_LPS) / Inflow_Rate_LPS) * 100
Critical Rupture: Pressure < 1.6 bar AND Loss > 22% -> Score 0.94 (Immediate Valve Shutoff)
Pressure Drop: Pressure < 2.0 bar AND Loss > 15% -> Score 0.78 (Acoustic Leak Inspection)
Location: backend/services/water_ai_service.js | backend/services/water_intelligence_service.js
```

* 6. Clinical RAG Hybrid Lexical-Semantic Search: Evidence-based clinical decision support querying ICMR, MoHFW, AIIMS, and WHO guidelines.

```
Step 1: Multilingual tokenization + 3-character n-gram shingles (handles Hinglish typos)
Step 2: BM25 / TF-IDF token overlap scoring + Keyword (+6.0) + Topic Boost (+8.0)
Step 3: Score normalization (>= 0.15 threshold) + SHA-256 Checksum Verification
Location: backend/services/clinical_rag_service.js | backend/services/clinical_care_assistant_service.js
```

* 7. Multi-Criteria Priority Scoring & SLA Ladder: Determines incident urgency and escalates delinquent requests automatically.

```
SLA Deadlines: CRITICAL = 2 Hours, HIGH = 6 Hours, MEDIUM = 24 Hours, LOW = 72 Hours
Escalation Daemon: Ticks every 60s, queries overdue requests, sets status = 'Escalated'
Emits WebSocket alerts: 'request:escalated' and 'notification:new'
Location: backend/services/priority_engine.js | backend/services/sla_engine.js
```

* 8. Computer Vision & ANPR Staging Workflow: Vehicle detection, license plate OCR, and violation staging.

```
Plate Pattern Validation: ^[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}$ (e.g. UP53BZ4821)
Human-in-the-Loop Guardrail: Violations staged as 'AI_FLAGGED'; require staff sign-off
Location: backend/services/cv_anpr_service.js | ai_service/camera_vision.py
```

* 9. Machine Learning Predictive Pipelines: Offline and online Scikit-Learn models with joblib persistence.

```
Traffic Congestion: RandomForestRegressor with TimeSeriesSplit cross-validation
Waste Priority: GradientBoostingClassifier (Normal, Medium, High, Immediate Dispatch)
Clinical Intent: TfidfVectorizer + Calibrated Logistic Regression (0% Emergency False Negatives)
Location: ai_service/artifacts/models/*.joblib | ai_service/training/*.py
```

* 10. Ultrasonic Time-of-Flight Firmware Physics: IoT dustbin depth measurement.

```
Sound Velocity in Air: 343 m/s = 0.0343 cm/microsecond
Distance (cm) = (Echo_Duration_us * 0.0343) / 2.0
Fill Percentage (%) = ((Total_Bin_Depth_cm - Distance) / Total_Bin_Depth_cm) * 100
PIR Interrupt: Footfall deposit increment via IRAM_ATTR ISR on pin change
Location: iot_hardware/esp32_smart_dustbin.ino
```

3. CORE MODULES & FUNCTION INVENTORY

The table below catalogs key functions across core backend services and their operational responsibilities:

* backend/services/traffic_engine.js:

* start() / stop(): Initializes 1s signal phase timer and 4s simulated camera telemetry tick.

* tickSignalCycles(): Manages countdowns, transitions Green->Yellow->Red, checks phase conflict interlocking.

* updateSimulatedTelemetry(): Applies natural vehicle flow jitter (+-7 veh/min) and triggers congestion updates.

* recalculateJunctionCongestionFromCameras(): Computes Level of Service (LOS) volume-to-capacity and speed drop.

* dispatchEmergencyCorridor(corridorId): Activates priority route, sets mode='Emergency Corridor', locks transit Green.

* deactivateEmergencyCorridor(corridorId): Restores junctions to 'Operational' AI Adaptive mode.

* backend/services/gorakhpur_road_graph.js:

* findNearestNode(queryOrCoords): Resolves landmark names or GPS coordinates to topological road network nodes.

* calculateThreeRoutes(origin, destination): Computes Fastest, Low-Traffic, and Min-Junction paths simultaneously.

* _dijkstra(startId, targetId, costFn): Generic Dijkstra priority-queue shortest-path solver.

* backend/services/clinical_care_assistant_service.js:

* detectLanguage(text): Analyzes Devanagari Unicode range [\u0900-\u097F] and Hinglish markers to select Hindi/English.

* extractMedicineFromQuery(text): Fuzzy regex normalizer for brand/generic medicines (paracetamol, metformin).

* triageEmergency(text): Evaluates acute cardiac, stroke, trauma, or respiratory red-flags.

* generateClinicalResponse(): Synthesizes 7-part comprehensive patient guidance with MySQL database grounding.

* backend/services/water_intelligence_service.js:

* haversineMeters(lat1, lon1, lat2, lon2): Calculates sphere geodesic distance.

* distToSegmentMeters(pLat, pLng, aLat, aLng, bLat, bLng): Calculates point-to-pipeline perpendicular distance.

* verifyWaterQualityAgainstRegulations(): Validates BOD, COD, TSS, pH, and Coliform against CPCB standards.

4. SOCKET.IO REAL-TIME CHANNELS & DAEMONS

Socket.IO handles bi-directional messaging with JWT handshake authentication and room segregation:

* Rooms: ambulance-tracking, parking-updates, emergency-alerts, water-updates, waste-updates, city-updates

* Traffic Signal Update: traffic:signal_update (emitted every 1s on phase change) | traffic:telemetry_tick (every 4s)

* Ambulance Radar Tracking: ambulance:location-updated (emitted every 2s with lat, lng, speed, bearing, battery)

* Green Wave Alerts: traffic:ambulance_green_wave | traffic:emergency_corridor_active | traffic:emergency_corridor_cleared

* Emergency SOS Ticker: emergency:new-sos (instant high-priority broadcast) | emergency:resolved

* Civic SLA Ticker: request:escalated (emitted by 60s SLA daemon when deadline is breached) | notification:new
* IoT Dustbin Updates: waste:bin-updated (triggered on ESP32 sensor ingest) | waste:vehicle-location
* Hospital Bed Telemetry: hospital:bed-updated | bed_status_changed (real-time ward occupancy sync)

5. REST API ENDPOINT CATALOG (29 ROUTE MODULES)

All APIs follow RESTful conventions, prefixed with /api, and secured via security middlewares:

* Authentication (/api/auth): POST /api/register, POST /api/login, POST /api/staff-login, POST /api/auth/otp/send, POST /api/auth/otp/verify, GET /api/auth/me, PUT /api/auth/profile, GET /api/user/activities
* Traffic & ATCS (/api/traffic): GET /api/traffic/junctions, GET /api/traffic/signals, POST /api/traffic/optimize-signal, POST /api/traffic/signals/override, GET /api/traffic/alternative-routes, GET /api/traffic/cameras, GET /api/traffic/violations/search, POST /api/traffic/violations/:id/dispute
* Emergency & Ambulances (/api/emergency, /api/ambulances): POST /api/emergency/sos, GET /api/emergency/incidents, PUT /api/emergency/incidents/:id/status, GET /api/ambulances, POST /api/ambulances/:id/location, POST /api/traffic/ambulance-preemption
* Hospital & Health (/api/hospital, /api/patients, /api/doctors): GET /api/hospitals, GET /api/hospital/beds, GET /api/hospital/bed-surge, POST /api/appointments/book-strict, GET /api/patients/:id/qr, POST /api/patients/verify-qr, POST /api/doctor/consultation
* Smart Parking (/api/parking): GET /api/parking/lots, GET /api/parking/slots, POST /api/parking/book, GET /api/parking/my-bookings, POST /api/parking/entry, POST /api/parking/exit
* Smart Waste (/api/waste): GET /api/waste/bins, POST /api/waste/prediction, POST /api/waste/optimize-route, POST /api/waste/report, POST /api/waste/dustbin-terminal/sensor-data
* SCADA Water (/api/water): GET /api/water/tanks, GET /api/water/pipelines, GET /api/water/anomalies, GET /api/water/schedules, POST /api/water/tanker-bookings
* AI Assistant & RAG (/api/ai): POST /api/ai/chat, POST /api/ai/assistant, POST /api/ai/clinical-care/chat, GET /api/ai/predictions
* Public Safety & Tourism (/api/police, /api/famous-places): GET /api/police/stations, POST /api/police/complaints, GET /api/famous-places, GET /api/famous/reviews
* ICCC Command Center (/api/admin): GET /api/admin/command-center, GET /api/admin/stats, POST /api/admin/what-if-simulation

6. TECHNOLOGY STACK & SECURITY SPECIFICATIONS

Frontend: HTML5, CSS3 Glassmorphic Token System, Vanilla ES6+ JS, Leaflet 1.9.4, OpenLayers 9.2.4, Socket.IO Client 4.8.1, QRCode.js, Capacitor 8.5 (Android APK).

Backend: Node.js 20+, Express.js 5.2.1, Socket.IO 4.8.3, mysql2/promise (Connection Pool: 20 conns), jsonwebtoken (JWT), zlib HTTP compression, multer, helmet, nodemailer.

AI & Machine Learning: Python 3.13, FastAPI, Uvicorn, Scikit-learn (RandomForest, GradientBoosting, LogisticRegression), Pandas, NumPy, Joblib, OpenCV (cv2), Google Gemini 1.5 Flash.

Database: MySQL 8.0+ Enterprise, 117 tables, utf8mb4_unicode_ci charset, ACID compliant transactions.

Security & Privacy: DPDP Act PII redaction (Aadhaar, PAN, Cards, Passwords), in-memory token blacklist, rate limiters (authRateLimiter: 10 req/15m, sosRateLimiter: 5 req/1m, aiRateLimiter: 30 req/1m), inputSanitizer XSS protection.